

Terzo Assignment - LICM

Generato da Doxygen 1.15.0

1 Gerarchia delle directory	1
1.1 Directory	1
2 Indice dei namespace	2
2.1 Lista dei namespace	2
3 Indice della gerarchia	2
3.1 Gerarchia delle classi	2
4 Indice dei tipi composti	2
4.1 Elenco dei tipi composti	2
5 Indice dei file	2
5.1 Elenco dei file	2
6 Documentazione delle directory	2
6.1 Riferimenti per la directory test	2
7 Documentazione dei namespace	3
7.1 Riferimenti per il namespace <code>anonymous_namespace{LoopPass.cpp}</code>	3
7.1.1 Documentazione delle funzioni	3
8 Documentazione delle classi	6
8.1 Riferimenti per la struct <code>anonymous_namespace{LoopPass.cpp}::LoopPass</code>	6
8.1.1 Documentazione delle funzioni membro	7
9 Documentazione dei file	8
9.1 Riferimenti per il file <code>LoopPass.cpp</code>	8
9.1.1 Descrizione dettagliata	9
9.1.2 Documentazione delle funzioni	9
9.2 Riferimenti per il file <code>test/LICM.cpp</code>	9
9.2.1 Documentazione delle funzioni	9
9.3 Riferimenti per il file <code>test/loop.cpp</code>	10
9.3.1 Documentazione delle funzioni	10
Indice analitico	11

1 Gerarchia delle directory

1.1 Directory

test	2
LICM.cpp	9
loop.cpp	10

2 Indice dei namespace

2.1 Lista dei namespace

Questa è l'elenco di tutti i namespace con una loro breve descrizione:

[anonymous_namespace{LoopPass.cpp}](#) 3

3 Indice della gerarchia

3.1 Gerarchia delle classi

Questo elenco di ereditarietà è ordinato approssimativamente, ma non completamente, in ordine alfabetico:

PassInfoMixin

[anonymous_namespace{LoopPass.cpp}::LoopPass](#) 6

4 Indice dei tipi composti

4.1 Elenco dei tipi composti

Queste sono le classi, le struct, le union e le interfacce con una loro breve descrizione:

[anonymous_namespace{LoopPass.cpp}::LoopPass](#) 6

5 Indice dei file

5.1 Elenco dei file

Questo è un elenco di tutti i file con una loro breve descrizione:

[LoopPass.cpp](#)
Implementazione del pass per la Loop-Invariant Code Motion 8

[test/LICM.cpp](#) 9

[test/loop.cpp](#) 10

6 Documentazione delle directory

6.1 Riferimenti per la directory test

File

- file [LICM.cpp](#)
- file [loop.cpp](#)

7 Documentazione dei namespace

7.1 Riferimenti per il namespace `anonymous_namespace{LoopPass.cpp}`

Composti

- struct [LoopPass](#)

Funzioni

- static bool [isInstructionInvariant](#) (Loop *L, Instruction *I, SmallPtrSetImpl< Instruction * > &LoopInvariant←
Ins)
Controlla se un'istruzione è loop-invariant.
- static bool [dominatesAllExits](#) (Instruction *I, DominatorTree &DT, const SmallVectorImpl< BasicBlock * >
&exitBlocks)
Verifica se l'istruzione candidata alla code motion domina tutte le uscite del loop.
- static bool [isDeadAtLoopExit](#) (Instruction *I, Loop *L)
Funzione che controlla se il valore definito dall'istruzione corrente non viene mai utilizzato al di fuori del loop.
- static bool [invOperandsMoved](#) (Instruction *I, Loop *L)
Verifica che tutti gli operandi definiti all'interno del loop, siano già stati spostati nel preheader.
- static bool [isInstructionHoistable](#) (Loop *L, Instruction *I, DominatorTree &DT, const SmallVectorImpl<
BasicBlock * > &exitBlocks)
Controlla se è safe effettuare la code motion (o code hoisting).

7.1.1 Documentazione delle funzioni

`dominatesAllExits()`

```
bool anonymous_namespace{LoopPass.cpp}::dominatesAllExits (
    Instruction * I,
    DominatorTree & DT,
    const SmallVectorImpl< BasicBlock * > & exitBlocks) [static]
```

Verifica se l'istruzione candidata alla code motion domina tutte le uscite del loop.

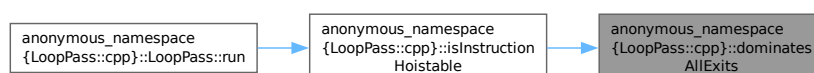
Parametri

<i>I</i>	puntatore all'istruzione candidata
<i>DT</i>	riferimento alla Dominator Tree
<i>exitBlocks</i>	vettore contenente i blocchi di uscita del loop

Restituisce

true se l'istruzione domina tutte le uscite, false altrimenti

Questo è il grafo dei chiamanti di questa funzione:



invOperandsMoved()

```
bool anonymous_namespace{LoopPass.cpp}::invOperandsMoved (
    Instruction * I,
    Loop * L) [static]
```

Verifica che tutti gli operandi definiti all'interno del loop, siano già stati spostati nel preheader.

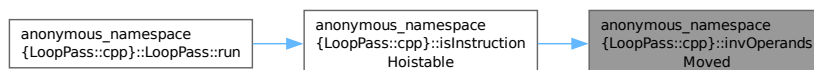
Parametri

<i>I</i>	puntatore all'istruzione considerata per lo spostamento
<i>L</i>	il loop corrente

Restituisce

true se gli operandi sono già stati spostati, false altrimenti

Questo è il grafo dei chiamanti di questa funzione:



isDeadAtLoopExit()

```
bool anonymous_namespace{LoopPass.cpp}::isDeadAtLoopExit (
    Instruction * I,
    Loop * L) [static]
```

Funzione che controlla se il valore definito dall'istruzione corrente non viene mai utilizzato al di fuori del loop.

Se l'istruzione non ha usi vivi all'esterno del loop, potremmo procedere con il code motion anche se l'istruzione non domina tutte le uscite del loop.

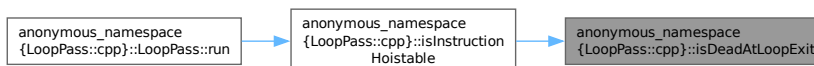
Parametri

<i>I</i>	puntatore all'istruzione
<i>L</i>	puntatore al loop corrente

Restituisce

true se il valore non viene mai usato fuori dal loop (dead at exit), false altrimenti

Questo è il grafo dei chiamanti di questa funzione:



isInstructionHoistable()

```
bool anonymous_namespace{LoopPass.cpp}::isInstructionHoistable (
    Loop * L,
    Instruction * I,
    DominatorTree & DT,
    const SmallVectorImpl< BasicBlock * > & exitBlocks) [static]
```

Controlla se è safe effettuare la code motion (o code hoisting).

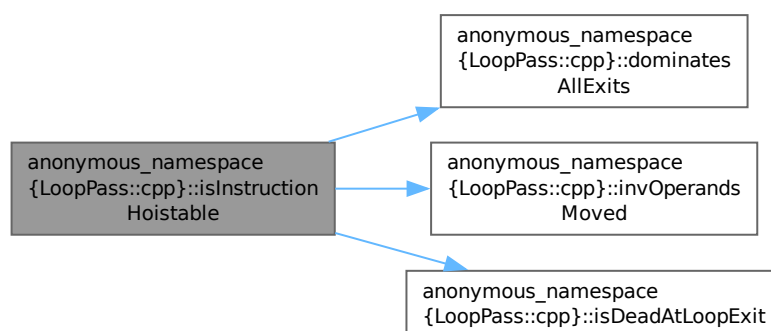
Parametri

<i>I</i>	puntatore all'istruzione candidata
<i>DT</i>	riferimento alla Dominator Tree
<i>exitBlocks</i>	vettore contenente i blocchi di uscita del loop

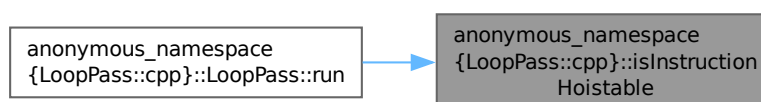
Restituisce

true se l'istruzione può essere spostata nel preheader, false altrimenti

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



isInstructionInvariant()

```
bool anonymous_namespace{LoopPass.cpp}::isInstructionInvariant (
    Loop * L,
    Instruction * I,
    SmallPtrSetImpl< Instruction * > & LoopInvariantIns) [static]
```

Controlla se un'istruzione è loop-invariant.

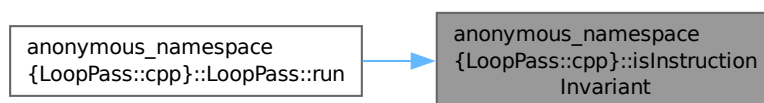
Parametri

<i>L</i>	
<i>I</i>	puntatore all'istruzione in esame
<i>LoopInvariantIns</i>	riferimento al set contenente le istruzioni già marcate come loop-invariant

Restituisce

true se l'istruzione è loop-invariant, false altrimenti

Questo è il grafo dei chiamanti di questa funzione:



8 Documentazione delle classi

8.1 Riferimenti per la struct `anonymous_namespace{LoopPass.cpp}::LoopPass`

Diagramma delle classi per `anonymous_namespace{LoopPass.cpp}::LoopPass`

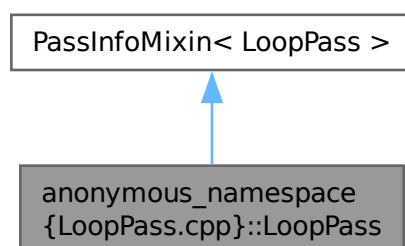
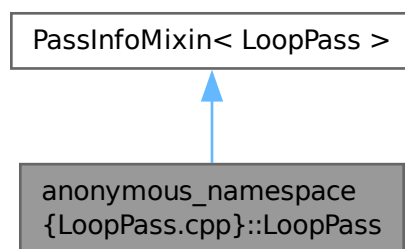


Diagramma di collaborazione per `anonymous_namespace{LoopPass.cpp}::LoopPass`:



Membri pubblici

- `PreservedAnalyses run` (`Function &F`, `FunctionAnalysisManager &AM`)
Function pass che effettua tutte le trasformazioni sui loop.

8.1.1 Documentazione delle funzioni membro

`run()`

```
PreservedAnalyses anonymous_namespace{LoopPass.cpp}::LoopPass::run (
    Function & F,
    FunctionAnalysisManager & AM) [inline]
```

Function pass che effettua tutte le trasformazioni sui loop.

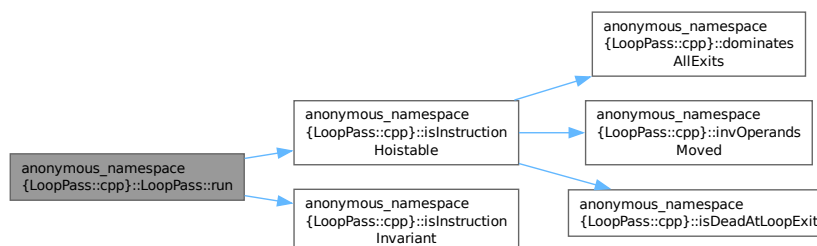
Parametri

<i>F</i>	Funzione su cui applicare i passi di trasformazione
<i>AM</i>	New pass manager per funzioni

Restituisce

Un oggetto `PreservedAnalyses` che indica quali analisi sono rimaste valide dopo il pass

Questo è il grafo delle chiamate per questa funzione:



La documentazione per questa struct è stata generata a partire dal seguente file:

- [LoopPass.cpp](#)

9 Documentazione dei file

9.1 Riferimenti per il file LoopPass.cpp

Implementazione del pass per la Loop-Invariant Code Motion.

```
#include <algorithm>
#include "llvm/IR/Function.h"
#include "llvm/IR/Instructions.h"
#include "llvm/IR/Constants.h"
#include "llvm/Analysis/LoopInfo.h"
#include "llvm/Passes/PassBuilder.h"
#include "llvm/Passes/PassPlugin.h"
#include "llvm/Support/raw_ostream.h"
#include "llvm/IR/Dominators.h"
#include "llvm/ADT/DepthFirstIterator.h"
#include "llvm/ADT/SmallPtrSet.h"
```

Grafo delle dipendenze di inclusione per LoopPass.cpp:



Composti

- struct [anonymous_namespace{LoopPass.cpp}::LoopPass](#)

Namespace

- namespace [anonymous_namespace{LoopPass.cpp}](#)

Funzioni

- static bool [anonymous_namespace{LoopPass.cpp}::isInstructionInvariant](#) (Loop *L, Instruction *I, SmallPtrSetImpl< Instruction * > &LoopInvariantIns)
Controlla se un'istruzione è loop-invariant.
- static bool [anonymous_namespace{LoopPass.cpp}::dominatesAllExits](#) (Instruction *I, DominatorTree &DT, const SmallVectorImpl< BasicBlock * > &exitBlocks)
Verifica se l'istruzione candidata alla code motion domina tutte le uscite del loop.
- static bool [anonymous_namespace{LoopPass.cpp}::isDeadAtLoopExit](#) (Instruction *I, Loop *L)
Funzione che controlla se il valore definito dall'istruzione corrente non viene mai utilizzato al di fuori del loop.
- static bool [anonymous_namespace{LoopPass.cpp}::invOperandsMoved](#) (Instruction *I, Loop *L)
Verifica che tutti gli operandi definiti all'interno del loop, siano già stati spostati nel preheader.
- static bool [anonymous_namespace{LoopPass.cpp}::isInstructionHoistable](#) (Loop *L, Instruction *I, DominatorTree &DT, const SmallVectorImpl< BasicBlock * > &exitBlocks)
Controlla se è safe effettuare la code motion (o code hoisting).
- llvm::PassPluginLibraryInfo [getLoopPassPluginInfo](#) ()
- LLVM_ATTRIBUTE_WEAK::llvm::PassPluginLibraryInfo [llvmGetPassPluginInfo](#) ()

9.1.1 Descrizione dettagliata

Implementazione del pass per la Loop-Invariant Code Motion.

9.1.2 Documentazione delle funzioni

getLoopPassPluginInfo()

```
llvm::PassPluginLibraryInfo getLoopPassPluginInfo ()
```

Questo è il grafo dei chiamanti di questa funzione:



llvmGetPassPluginInfo()

```
LLVM_ATTRIBUTE_WEAK::llvm::PassPluginLibraryInfo llvmGetPassPluginInfo ()
```

Questo è il grafo delle chiamate per questa funzione:



9.2 Riferimenti per il file test/LICM.cpp

Funzioni

- int [LICM](#) ()

9.2.1 Documentazione delle funzioni

LICM()

```
int LICM ()
```

9.3 Riferimenti per il file test/loop.cpp

Funzioni

- int `fun` (int a, int b)

9.3.1 Documentazione delle funzioni

`fun()`

```
int fun (  
    int a,  
    int b)
```

Indice analitico

- anonymous_namespace{LoopPass.cpp}, [3](#)
 - dominatesAllExits, [3](#)
 - invOperandsMoved, [3](#)
 - isDeadAtLoopExit, [4](#)
 - isInstructionHoistable, [4](#)
 - isInstructionInvariant, [5](#)
- anonymous_namespace{LoopPass.cpp}::LoopPass, [6](#)
 - run, [7](#)
- dominatesAllExits
 - anonymous_namespace{LoopPass.cpp}, [3](#)
- fun
 - loop.cpp, [10](#)
- getLoopPassPluginInfo
 - LoopPass.cpp, [9](#)
- invOperandsMoved
 - anonymous_namespace{LoopPass.cpp}, [3](#)
- isDeadAtLoopExit
 - anonymous_namespace{LoopPass.cpp}, [4](#)
- isInstructionHoistable
 - anonymous_namespace{LoopPass.cpp}, [4](#)
- isInstructionInvariant
 - anonymous_namespace{LoopPass.cpp}, [5](#)
- LICM
 - LICM.cpp, [9](#)
- LICM.cpp
 - LICM, [9](#)
- llvmGetPassPluginInfo
 - LoopPass.cpp, [9](#)
- loop.cpp
 - fun, [10](#)
- LoopPass.cpp, [8](#)
 - getLoopPassPluginInfo, [9](#)
 - llvmGetPassPluginInfo, [9](#)
- Riferimenti per la directory test, [2](#)
- run
 - anonymous_namespace{LoopPass.cpp}::LoopPass, [7](#)
- test/LICM.cpp, [9](#)
- test/loop.cpp, [10](#)